

Smart Barangay AI

Requirements Audit Report

Audit date: 31 August 2026
Audited branch: `fix/sync-admin-resident-supabase`, synchronized with `origin/main` at `0d23a14`
Scope: Resident PWA, Admin Portal, API, and Supabase schema
Assessment rule: A feature is considered existing only when it is persisted and works end-to-end, not merely displayed in a mock or static screen.

5 Partial / broken	1 Missing	1 Implemented
-----------------------	--------------	------------------

Requirement Matrix

Requirement	Status	Evidence and audit finding
Chatbot: dynamic form	Partial	The chatbot detects document keywords and opens an in-chat form. The form only collects a purpose/notes field. It has no dynamic service-specific fields, document uploads, requirements validation, or structured form data. <code>apps/resident-pwa/components/chat/InChatFormCard.tsx:22-53</code>
Admin receives requests; one transaction/chatbot session for multiple requests	Partial / broken in real DB	The Admin Portal has a request queue and status actions. However, requests are not grouped to a chatbot session or transaction. The frontend creates a session ID but saves chat messages with <code>session_id: null</code> . The request form submits the auth user ID as <code>resident_id</code> , while the database expects a residents-table ID. <code>apps/resident-pwa/app/(app)/requests/new/page.tsx:52-61</code> <code>supabase_schema.sql:91-99</code>
Notify users when documents are ready for pickup, including payment requirements	Partial	A resident notification inbox exists and Admin can mark a request as Released. There is no automatic notification when a status changes. There are also no payment amount, due status, receipt, or payment-required fields in the request/notification model. <code>apps/admin-portal/app/(dashboard)/documents/actions.ts:55-90</code> <code>supabase_schema.sql:206-221</code>
Admin generates a filled template from chatbot data	Missing	The documents table only stores a file URL and generating user. No request form payload, document templates, field mapping, or PDF/DOCX generation workflow exists. <code>supabase_schema.sql:102-107</code>
Requests recorded for dashboard reports	Partial	Document requests are stored and the dashboard counts pending requests. Monthly transaction chart data is hard-coded, and the Reports page displays static mock reports rather than generating or downloading reports from database records. <code>apps/admin-portal/app/(dashboard)/dashboard/actions.ts:28-76</code>
User registration requires staff review, validation, and approval	Implemented	The latest main branch now links the auth user to the resident profile, saves the address, assigns the Resident role, restricts pending/rejected accounts, loads pending residents from the database, supports staff-permission checks, approve/reject actions, rejection reasons, audit logging, and realtime queue refresh. The new migration must be applied to the deployed Supabase project. Applicant notifications after approval/rejection are still absent. <code>apps/resident-pwa/components/auth/RegisterForm.tsx:61-137</code> <code>apps/admin-portal/app/(dashboard)/residents/actions.ts:141-268</code> <code>supabase/migrations/001_registration_approval_rbac.sql:1-204</code>
Clickable predefined questions only when answers are available	Partial	Five welcome buttons are hard-coded. They are not dynamically filtered against available knowledge or answerability. The knowledge base includes additional topics such as Business Clearance and Events, but those are not included in the buttons. <code>apps/resident-pwa/components/chat/ChatInterface.tsx:363-373</code> <code>apps/resident-pwa/lib/ai/policy-knowledge.ts:12-64</code>

Capstone / Thesis Alignment Review

The attached 79-page capstone, *Smart Barangay: An AI-Powered Web and Mobile Services Portal for Barangay Tandang Sora, Butuan City*, was reviewed as the system's intended specification. The following checks go beyond the original feature checklist.

Capstone claim	Alignment	Deeper finding
Two synchronized components: staff Web Portal and resident Mobile/PWA application	Partial	Both Next.js applications exist and share Supabase. Synchronization is not yet reliable for requests because of the resident-ID mismatch, and several admin modules are still mock/static screens. The resident application is a PWA, not a native Android/iOS application.
AI-powered multilingual chatbot using LLM, NLP, and RAG from approved barangay documents	Partial	A FastAPI/RAG path and multilingual controls exist, but the resident route can fall back to client/local keyword knowledge. The built-in fallback contains fixed policy answers, so production answerability must be tied to approved, versioned barangay sources only. The RAG/API deployment and source freshness need an integration test.
Session-based AI memory maintains context across turns	Not aligned	The frontend creates a random session ID but stores chat messages with a null session. The repository has both <code>chat_sessions/chat_messages</code> and <code>ai_sessions/ai_messages</code> models, with no single end-to-end session contract. Memory persistence is therefore unreliable.
Session privacy and deletion under the Data Privacy Act	Not aligned	The thesis says session data is isolated and permanently deleted when a session ends. The current app loads and persists chat history, while the backend has memory/audit persistence and no resident-facing end-session deletion flow. Define retention, deletion, audit minimization, and consent rules before claiming compliance.
Dynamic form generation inside chat, followed by automatic routing to staff	Partial / broken	Only a fixed purpose textarea is shown. The request payload lacks service-specific fields, uploads, validation, session linkage, and a durable routing/group identifier. Admin queue retrieval exists but the resident foreign-key path must be corrected.
Admin manages residents, senior citizens, documents, announcements, and reports	Partial	General resident CRUD and registration approval are real paths. Senior, PWD, household, certificate, business-clearance, and several community/report views still use mock arrays or local state. These cannot support capstone claims of synchronized operational records until connected to database actions.
Approval/rejection and notification workflow	Partial	Registration approval/rejection is now database-backed in the latest main branch. Document approval/release is database-backed, but neither workflow consistently creates resident notifications. PWA push/SMS delivery is not implemented; the current resident notification is in-app only.
Automated report generation and performance benchmarking	Missing	The report screen and monthly chart use static data. There is no report generation/export pipeline, senior citizen annual report implementation, or recorded benchmark for the thesis metrics: 200-query intent accuracy, report generation time, and transaction processing speed.
Role-based access control and secure resident records	Partial	RBAC and registration-related RLS were added in the latest main branch. However, the base schema documents RLS only for chat messages, and document request/data access policies are not comprehensively enforced at the database boundary. Server actions need permission checks and resident ownership checks, not UI-only protection.
Official Barangay Tandang Sora knowledge and operational data	Partial	The repository has seed policies and Tandang Sora references, but some demo/local knowledge contains fixed fees, schedules, events, and stock content. These must be marked as approved, versioned, locally verified, and auditable before they are presented as official data.

Required Work Before Claiming Capstone Completion

- P0 — Data correctness:** Create one canonical `resident_id` lookup by authenticated user, enforce ownership with RLS, and test request creation against a real Supabase database.
- P0 — Session/privacy contract:** Choose either `ai_sessions/ai_messages` or `chat_sessions/chat_messages`; persist the session correctly, link requests to it, implement end-session deletion/retention, and remove sensitive content from unnecessary audit logs.
- P0 — Request workflow:** Define service configuration and dynamic fields, required documents, validation, request groups, statuses, assignment, approval/rejection reason, pickup/delivery, and payment state in the database.
- P0 — Notifications:** Trigger in-app notifications for every status transition and add a real delivery adapter only if SMS/push is part of the approved capstone scope. Include amount, payment status, receipt, due date, and pickup instructions.
- P1 — Document automation:** Add approved templates, field mapping, generated PDF/DOCX output, secure storage, document versioning, and an audit trail.
- P1 — Operational admin modules:** Replace mock senior/PWD/household/certificate/business/report pages with database-backed actions and real filters.
- P1 — Reporting:** Generate request-volume, processing-time, status, demographic, senior-citizen, and payment reports from real records with CSV/PDF export.

8. **P1 — Answer system:** Make suggested questions derive from active approved knowledge/services and show a suggestion only when an answer source is available. Add source version, freshness, confidence, and escalation behavior.
9. **P1 — Evaluation evidence:** Add traceable tests for the thesis claims: 200 multilingual chatbot queries, request processing latency, report generation timing, privacy/authorization tests, and UAT task completion for residents and staff.
10. **P2 — Thesis wording:** Reconcile “Waterfall” versus “iterative sprints,” “mobile application” versus PWA, and “permanent deletion” versus retained chat/audit data so the written capstone matches the deployed behavior.

Minimum Acceptance Criteria

- A registered resident can submit a request and the exact same request appears in the authorized admin queue with the correct resident identity.
- A single chatbot session can create multiple related requests without losing context or mixing users.
- Approval, rejection, payment-required, and ready-for-pickup events create visible notifications for the correct resident.
- An authorized staff member can generate a completed official template from the submitted data and retrieve it securely.
- Every dashboard value and exported report can be traced to database records and a selected date range.
- Unauthenticated users, other residents, and unauthorized staff cannot read or mutate protected requests, documents, profiles, or sessions.

Highest-Priority Missing Work

1. Fix the authenticated-user to resident-profile relationship and use the correct resident ID for requests.
2. Implement a structured dynamic chatbot request form with service-specific fields, uploads, validation, and saved payloads.
3. Persist chatbot sessions and link multiple document requests to one session/transaction.
4. Add payment fields and automatic resident/admin notifications for approval, payment, and pickup readiness.
5. Connect registration review actions to the database, including validation, approval/rejection reason, and applicant notification.
6. Implement template generation and real dashboard/report queries using stored request data.

Validation Performed

Attached thesis PDF: successfully read and reviewed (79 pages).

Resident PWA TypeScript check: passed after the branch synchronization.

Admin Portal TypeScript check: passed after the branch synchronization.

Branch synchronization: fast-forwarded to `origin/main`; local work was restored and four conflicts were resolved while preserving both sides.

Type checking does not detect the runtime foreign-key mismatch or static/mock-only behavior identified in this audit.

Generated from the Smart Barangay AI repository audit. This report describes the current code state and does not modify application functionality.